지갑에서 수탁까지 :

사고 위험을 구조적으로 낮추는 자산보호 아키텍처

OT-OCN 기반 실행 경로 분리를 통한
구조적 보안 설계



www.smartcat.tech

Abstract

디지털 자산을 안전하게 보관한다는 것은 단순히 키를 금고에 넣는 문제가 아니다. 진짜 중요한 질문은 이것이다.

"누가, 어떤 조건에서, 어떤 근거로 자산을 움직일 수 있는가?"

지금까지의 수탁 시스템은 대부분 서버가 이 질문의 중심에 서 있었다. 서버가 요청을 받고, 서버가 판단하고, 서버가 서명을 요청하는 구조이다. 이 방식은 운영이 편리하지만, 서버가 침해되면 대규모 자산 손실로 이어질 수 있다.

반대로 모든 권한을 개인이 직접 들고 있는 자가수탁 방식은 보안은 강하지만, 기업이 요구하는 정책 통제, 감사, 승인 절차를 적용하기 어렵다.

본 백서는 이 두 방식의 장점을 결합한다.

- 사용자는 개인 콜드월렛으로 자신의 의사를 직접 서명하고
- 기업은 HSM을 통해 정책에 따라 자산을 통제하며
- 서버와 HSM 사이의 민감한 구간에는 OT-OCN 스타일 내부 전용 채널을 사용한다.

이 구조의 목표는 단순하다.

"서버가 침해되어도 자산이 움직이지 않는 수탁 시스템"

1. 문제 정의 (Problem Statement)

1.1 서버 중심 수탁 모델의 한계

대부분의 수탁 시스템에서 자산 이동 흐름은 다음과 같다.



이 구조에서 서버는 단순 중계자가 아니라, 사실상 승인 흐름의 중심이다. 서버가 공격받으면 공격자는 출금 요청을 위조하거나 정책을 우회할 수 있다.

많은 사고가 바로 이 지점에서 발생한다. 즉, 문제의 핵심은 네트워크가 아니라 권한의 위치에 있다.

1.2 완전 자가수탁 모델의 한계

자가수탁(Self-Custody) 모델에서는 사용자가 자신의 자산을 보호하기 위해 모든 개인 키를 직접 보관하고 관리한다. 이 방식은 중앙 시스템을 신뢰하지 않아도 된다는 점에서 강력한 보안적 장점을 가진다. 중앙 서버나 수탁 기관이 존재하지 않기 때문에, 단일 침해 지점(Single Point of Failure)이 없다는 점은 분명한 장점이다.

그러나 자가수탁 모델은 보안과는 다른 차원의 문제를 드러낸다. 사용자가 모든 권한을 직접 보유하는 구조에서는 기업이나 기관이 요구하는 정책을 적용하기가 사실상 불가능하다. 예를 들어, 일정 금액 이상의 자산 이동에 대해 추가 승인을 요구하거나, 특정 주소로의 전송을 제한하는 정책을 기술적으로 강제할 수 없다.

또한 자가수탁 환경에서는 승인 이력과 의사결정 과정이 체계적으로 기록되지 않는 경우가 많다. 누가 언제 어떤 근거로 자산 이동을 승인했는지에 대한 명확한 감사 기록을 남기기 어렵고, 사고가 발생했을 때 책임의 범위를 명확히 구분하기도 쉽지 않다.

이러한 이유로 자가수탁 모델은 개인 사용자에게는 적합할 수 있지만, 기관 수탁, 기업 재무 관리, 규제 환경에서는 그대로 적용하기 어렵다는 한계를 갖는다.

1.3 산업이 요구하는 방향

디지털 자산 산업이 성숙 단계로 접어들면서, 시장은 단순히 "키를 누가 가지고 있는가"라는 질문을 넘어서기 시작했다. 이제 산업이 요구하는 것은 보안, 사용성, 책임 구조가 동시에 충족되는 체계적인 수탁 모델이다.

첫째, 자산 이동에는 반드시 사용자의 명시적인 동의가 포함되어야 한다. 사용자가 인지하지 못한 상태에서 자산이 이동되는 구조는 어떤 경우에도 허용되어서는 안된다.

둘째, 동시에 기업과 기관의 정책이 자동으로 집행될 수 있어야 한다. 금액 한도, 주소 관리, 승인 절차, 쿨다운 기간과 같은 정책은 사람의 판단이나 운영 관행이 아니라 시스템 차원에서 강제되어야 한다.
셋째, 서버가 침해되더라도 자산 이동이 불가능한 구조가 필요하다. 서버는 언제든 공격의 대상이 될 수 있으며, 서버 보안을 전제로 한 수탁 모델은 근본적으로 불완전하다.

마지막으로, 모든 자산 이동과 승인 과정은 명확한 감사 기록으로 남아야 한다. 누가, 언제, 어떤 근거로 승인했는지를 사후에 재현할 수 있어야 하며, 이는 규제 대응과 신뢰 확보를 위해 필수적인 요소이다.

본 백서는 이러한 네 가지 요구사항—사용자 동의, 정책 자동 집행, 서버 침해 내성, 감사 가능성—을 동시에 만족하는 수탁 아키텍처를 제안하는 것을 목표로 한다.

2. 제안하는 해결책 개요

본 백서의 출발점은 단순하지만 분명합니다. **서버는 어떤 경우에도 자산 이동을 결정할 수 없어야 한다**는 원칙이다.

기존 수탁 구조에서는 서버가 승인 흐름의 중심에 위치하는 경우가 많다. 서버는 요청을 받고, 조건을 확인하고, 서명 시스템에 전달하는 역할을 수행하며, 결과적으로 자산 이동 여부에 실질적인 영향을 미친다. 그러나 서버는 외부 공격과 내부 운영 리스크에 가장 많이 노출되는 구성 요소이기도 하다.

본 시스템은 이 구조를 근본적으로 바꾼다. 자산 이동에 대한 판단과 책임을 하나의 주체에 집중시키지 않고, 다음 세 단계로 명확히 분리한다.

첫째, 사용자가 먼저 승인한다.
모든 자산 이동 요청은 사용자의 개인 콜드월렛 서명을 통해 시작된다.
이는 해당 거래가 사용자의 명시적인 의사에 의해 생성되었음을 증명하는 단계이다.

둘째, 서버는 요청을 전달한다.
서버는 인증, 형식 검증, 리스크 평가 등 운영상 필요한 역할을 수행하지만, 자산 이동을 허용하거나 거부하는 최종 권한을 가지지 않는다. 서버는 승인자가 아니라 중계자이다.

셋째, HSM만이 최종 실행을 담당한다.
모든 조건이 충족되었는지 확인한 후, 실제로 자산을 움직일 수 있는 유일한 주체는 HSM이다. HSM은 사용자 서명과 서버 서명, 그리고 사전에 정의된 정책을 모두 검증한 경우에만 최종 서명을 수행한다.

이 세 단계 중 어느 하나라도 충족되지 않으면, 자산은 움직이지 않는다.

이러한 원칙을 구현하기 위해 본 시스템은 자산 이동과 관련된 승인 주체를 명확히 세 가지로 분리한다. **사용자 개인 콜드월렛, 서버,** 그리고 **HSM 기반 수탁 클러스터**다. 이 중 오직 HSM만이 최종 실행 권한을 가지며, 다른 어떤 구성 요소도 단독으로 자산 이동을 완결할 수 없다.

또한 서버와 HSM 사이의 통신 구간은 일반적인 API 경로와 분리된 **OT-OCN 스타일의 내부 전용 채널**을 사용합니다. 이 채널에서는 실제 자산 정보나 의미 있는 요청 내용을 직접 전달하지 않습니다. 대신, 의미가 제거된 메타데이터 토큰(Name)만을 라우팅 키로 사용하며, 요청의 실제 의미 해석과 정책 판단은 오직 HSM 내부에서만 수행됩니다.

이를 통해 본 시스템은 서버 침해 상황에서도 자산 이동이 불가능한 구조를 유지하면서, 동시에 기관 수준의 정책 통제와 감사 가능성을 확보합니다.

3. 아키텍쳐 구성 요소

본 장에서는 본 백서에서 제안하는 시스템을 구성하는 각 요소가 어떤 역할을 수행하며, 자산 이동 과정에서 어떻게 상호작용하는지를 단계별로 설명합니다.

3.1 User / Mobile App

사용자는 모바일 애플리케이션을 통해 자산 출금 또는 송금 요청을 생성합니다. 이 과정에서 애플리케이션은 단순한 버튼 입력을 넘어, 하나의 "Tx-Intent(거래 의도 객체)"를 생성합니다.

Tx-Intent는 사용자가 실제로 수행하고자 하는 거래의 핵심 정보를 담은 구조화된 데이터로, 대상 주소, 전송 금액, 사용될 블록체인 네트워크, 그리고 요청의 유효시간(TTL) 등을 포함합니다.

필요한 경우, 사용자는 본인 인증이나 자격 증명을 위한 VC/VP를 함께 첨부할 수 있습니다. 이는 이후 정책 판단이나 감사 과정에서 활용될 수 있습니다.

중요한 점은 이 단계에서 **실제 자산 이동은 전혀 발생하지 않는다**는 것입니다. 사용자는 단지 "이러한 거래를 원한다"는 의사를 시스템에 전달하는 것에 불과합니다.

3.2 Personal Cold Wallet

Tx-Intent가 생성되면, 사용자의 개인 콜드월렛이 개입한다. 개인 콜드월렛은 하드웨어 지갑이나 보안 칩과 같이 사용자가 직접 소유·통제하는 장치입니다.

이 구성 요소는 두 가지 역할을 수행합니다.

첫째, 로그인 또는 세션 생성을 위한 챌린지 서명을 통해 사용자의 접근 권한을 증명합니다.

둘째, Tx-Intent 자체에 대한 서명을 수행합니다.

이때 생성되는 서명이 바로 **user_cold_sig**입니다. 이 서명은 "이 거래는 사용자가 명시적으로 승인했다"는 책임 서명이며, 본 시스템에서 자산 이동의 출발점이 되는 가장 중요한 증거입니다.

HSM은 user_cold_sig가 포함되지 않은 어떤 요청도 실행하지 않습니다. 즉, 사용자 승인 없이 자산이 이동하는 경로는 구조적으로 차단됨을 의미합니다.

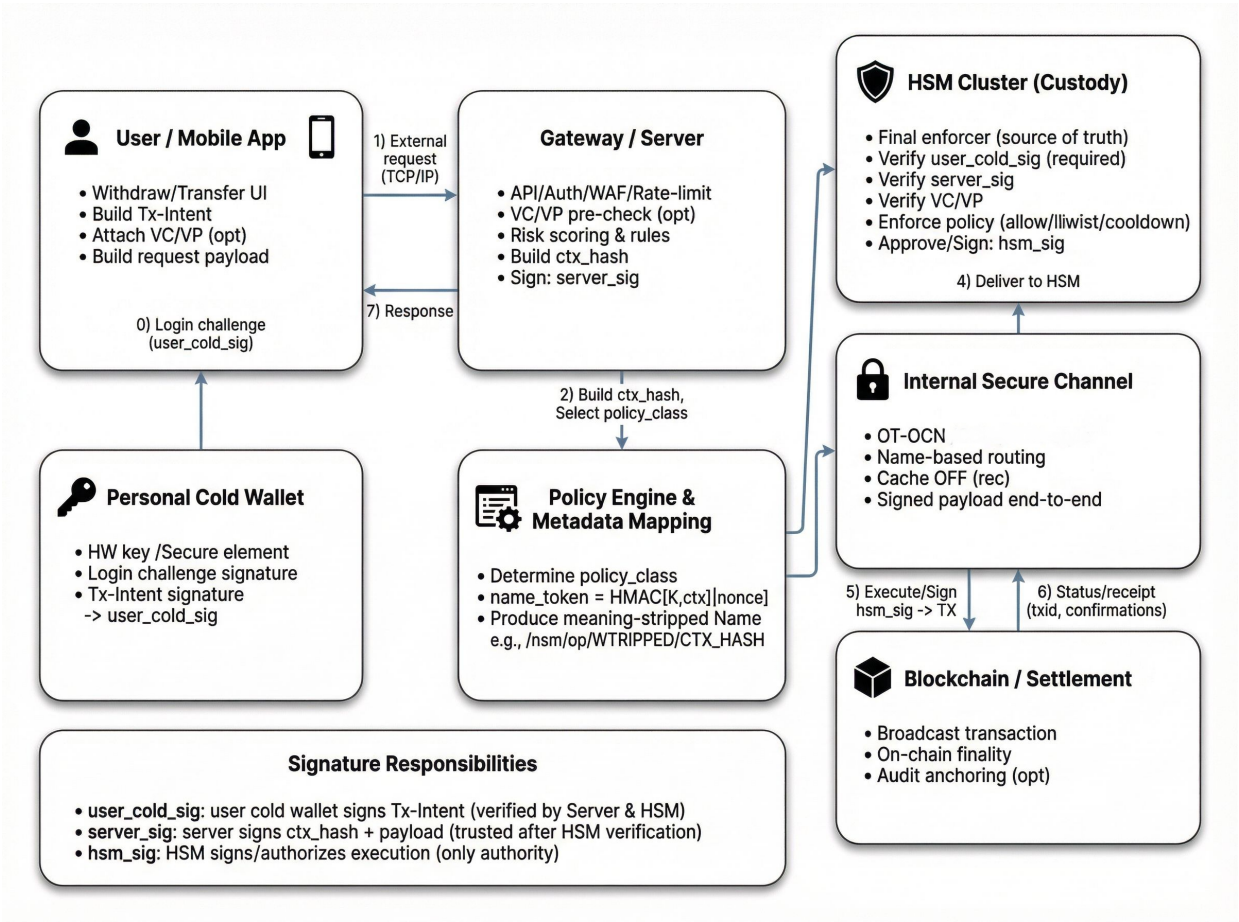
3.3 Gateway / Server

서버는 외부 요청을 받아 내부 시스템으로 전달하는 **중계자 역할**을 수행합니다. 서버는 다음과 같은 운영적 기능을 담당합니다.

- 사용자 인증 및 접근 제어
- WAF 및 Rate Limit을 통한 공격 방어
- 요청 형식 및 기본 무결성 검증
- 이상 징후를 감지하기 위한 리스크 스코어 계산

또한 서버는 요청 전체를 요약한 컨텍스트 해시(ctx_hash)를 생성하고, 해당 요청이 정상적인 프로세스를 통해 전달되었음을 증명하기 위해 **server_sig**를 생성합니다.

중요한 점은 서버가 **자산 이동을 승인하거나 결정하지 않는다**는 것입니다. 서버는 요청을 전달할 뿐이며, 최종 실행 여부에 대한 권한을 가지지 않습니다.



3.4 Policy Engine & Metadata Mapping

서버는 외부 요청을 그대로 내부 시스템에 전달하지 않는다. 대신, 요청을 내부 전달에 적합한 형태로 변환하는 과정을 거친다.

먼저 서버는 요청의 성격을 분석하여 정책 클래스를 결정한다. 예를 들어 해당 요청이 출금인지, 내부 전송인지, 주소 등록인지에 따라 서로 다른 정책이 적용된다.

정책 클래스 결정

예:

- WITHDRAW
- TRANSFER
- ADDRESS_REGISTER

그 다음, 서버는 외부 요청의 의미를 제거한 내부 라우팅용 Name을 생성한다. 이 Name은 다음과 같은 방식으로 만들어진다.

의미 제거 Name 생성

ini: name_token = HMAC(K_shared, policy_class || ctx_hash || nonce)

예:

swift: /hsm/op/WITHDRAW/CTX_ab31f9

이 Name에는 주소, 금액, DID 등 민감한 정보가 포함되지 않는다. 실제 의미를 담은 데이터는 암호화된 payload로 전달되며, 그 해석은 오직 HSM 내부에서만 이루어진다.

3.5 OT-OCN Style Internal Secure Channel

서버에서 생성된 내부 요청은 **OT-OCN 스타일의 전용 내부 채널**을 통해 HSM으로 전달된다. 이 채널은 외부 인터넷 API 경로와 완전히 분리된 통신 경로이다.

이 내부 채널은 다음과 같은 특징을 가진다.

- Name 기반 라우팅을 사용
- 중간 캐시를 사용하지 않음
- 종단간 암호화(mTLS)를 적용
- 메시지 단위 서명을 통한 무결성 보장

이 채널의 목적은 인터넷을 대체하는 것이 아니다. 목적은 오직 하나, **HSM으로 향하는 실행 경로를 일반 API 경로와 분리하는 것이다.**

이를 통해 서버 침해나 외부 공격이 곧바로 HSM 실행으로 이어지는 것을 방지한다.

3.6 HSM Cluster (Custody)

HSM 클러스터는 본 시스템에서 자산 이동의 최종 집행자이다. 실제로 자산을 움직일 수 있는 유일한 권한은 HSM에만 존재한다.

HSM은 내부 요청을 수신하면 다음 항목들을 순차적으로 검증한다.

- user_cold_sig의 유효성
- server_sig의 정합성
- 필요 시 VC/VP의 유효성
- 사전에 정의된 정책 엔진 평가

정책 엔진은 다음과 같은 조건을 판단할 수 있다.

- 전송 금액이 허용된 한도를 초과하는지 여부
- 대상 주소가 화이트리스트에 포함되어 있는지
- 신규 주소에 대한 쿨다운 기간 적용 여부
- 다중 승인(M-of-N)이 필요한 상황인지 여부

모든 조건을 충족한 경우에만, HSM은 자신의 키로 hsm_sig를 생성하고 이를 기반으로 실제 트랜잭션을 생성한다.

3.7 Blockchain / Settlement

HSM이 생성한 트랜잭션은 해당 블록체인 네트워크로 전파되고, 네트워크의 합의 과정을 거쳐 최종적으로 확정된다. 이로써 자산 이동은 완료되며, 이 과정 전반에 대한 승인 이력과 서명 증거는 감사 및 사후 검증을 위해 보존된다.

요약: 이 아키텍처에서 중요한 점은 단 하나다.

어느 구성 요소도 단독으로 자산을 움직일 수 없으며, 사용자 승인, 서버 중계, HSM 정책 집행이 모두 결합되어야만 실행이 가능하다.

---

4. 메타데이터 기반 매핑의 보안 의미

본 구조에서 메타데이터는 단순한 부가 정보가 아니라, 시스템의 보안 수준을 결정짓는 핵심 요소로 다루어진다. 따라서 메타데이터는 처음부터 "어떻게 보호할 것인가"가 아니라, "왜 외부로 노출될 필요가 없는가"라는 관점에서 설계된다.

먼저, 외부에서 들어오는 요청이 가진 의미 "예를 들어 전송 대상 주소, 금액, 사용자 식별자와 같은 민감한 정보"는 내부 통신에 그대로 전달되지 않는다. 이러한 외부 의미는 서버 단계에서 분리되며, 내부 채널에서는 해당 의미를 직접 포함하지 않는 형태로 요청이 재구성된다.

내부 채널에서 사용되는 Name은 실제 의미를 담는 식별자가 아니라, 단지 요청을 올바른 처리 지점으로 전달하기 위한 라우팅용 토큰이다. 이 Name은 정책 클래스와 컨텍스트 해시를 기반으로 생성되며, 그 자체만으로는 자산 이동의 세부 내용을 추론할 수 없다.

실제 요청의 의미 해석과 정책 판단은 오직 HSM 내부에서만 이루어진다. 외부 시스템이나 중간 전달 경로에서는 요청이 무엇을 의미하는지 알 필요도 없고, 알 수도 없도록 설계된다. 이로써 의미 해석 권한은 자연스럽게 HSM으로 집중된다.

이러한 구조 덕분에, 설령 내부 채널에서 패킷이 노출되는 상황이 발생하더라도 해당 패킷만으로는 요청의 목적이나 내용을 이해할 수 없다. 민감한 정보는 암호화된 payload의 형태로만 존재하며, 이를 해석할 수 있는 권한은 HSM에 한정된다.

결과적으로 본 시스템은 메타데이터를 평문 상태로 보호하려는 접근을 취하지 않는다. 대신, 메타데이터가 평문으로 외부에 노출될 이유 자체를 제거함으로써 보안을 달성한다. 이는 "정보를 숨기는 보안"이 아니라, "정보가 흘러다닐 필요가 없는 구조적 보안"에 가깝다.

5. 위협 모델과 대응

본 시스템은 보안을 특정 네트워크 기술이나 단일 장비의 신뢰에 의존하지 않는다. 대신, 실제 수탁 및 디지털 자산 관리 환경에서 발생 가능한 공격 시나리오를 전제로 하여, 각 위협이 자산 이동으로 이어지지 않도록 **구조적으로 분리된 대응 방식**을 채택한다. 본 장에서는 대표적인 위협 유형을 기준으로 시스템의 기본 대응 원칙을 설명하고, 이어서 내부 실행 채널까지 침해된 상황을 가정한 심화 위협 분석을 제시한다.

5.1 대표적 위협 시나리오와 기본 대응 구조

첫 번째 위협은 **서버 침해이다**.
서버는 외부 네트워크와 직접 연결된 구성요소로, 공격자가 가장 먼저 노리는 대상이 된다. 기존 수탁 구조에서는 서버 침해가 곧 자산 이동으로 이어질 수 있다. 그러나 본 시스템에서 서버는 자산 이동의 승인 주체가 아니다. 서버가 침해되더라도, 사용자 개인 콜드월렛에서 생성된 user_cold_sig가 없으면 어떠한 자산 이동도 실행될 수 없다. 서버는 요청을 전달할 수 있을 뿐, 사용자의 명시적 승인을 대체하거나 위조할 수 없다.

두 번째 위협은 **사용자 개인 키의 탈취이다**.
개인 키 유출 가능성을 완전히 배제하는 것은 현실적으로 불가능하다. 이를 전제로 본 시스템은 개인 키 하나만으로 자산 이동이 불가능하도록 설계된다. HSM은 사전에 정의된 정책 엔진을 통해 전송 금액 한도, 주소 화이트리스트, 신규 주소에 대한 쿨다운 기간, 다중 승인 요구 여부 등을 평가한다. 따라서 사용자 키가 탈취되더라도, 정책 조건을 충족하지 못하는 요청은 자동으로 제한되거나 보류된다.

세 번째 위협은 **내부자에 의한 오남용이다.**
운영자나 내부 시스템 접근 권한을 가진 사용자가 의도적이거나 실수로 자산 이동을 시도하는 경우도 현실적인 위협이다. 본 구조에서는 내부자가 단독으로 자산 이동을 실행할 수 있는 권한을 갖지 않는다. 모든 자산 이동은 HSM 정책 엔진을 통과해야 하며, 그 과정은 감사 로그로 기록된다. 이를 통해 내부자의 행위는 사후 추적 가능하며, 책임 소재를 명확히 할 수 있다.

네 번째 위협은 **통신 구간에서의 패킷 스니핑이다.**
서버와 HSM 사이의 내부 통신 구간에서 트래픽이 노출되는 상황을 가정하더라도, 본 시스템에서는 해당 트래픽만으로 요청의 의미를 파악할 수 없다. 내부 채널에서는 의미가 제거된 Name만이 라우팅 키로 사용되며, 실제 자산 정보는 암호화된 payload 형태로 전달된다. 따라서 패킷이 관찰되더라도 요청의 목적이나 내용을 추론하는 것은 구조적으로 제한된다.

이와 같이 본 시스템은 개별 위협을 단편적으로 차단하는 방식이 아니라, 서명 분리, 정책 강제, 실행 권한 집중, 의미 제거 통신이라는 구조적 원칙을 통해 위협이 자산 이동으로 직접 연결되지 않도록 설계된다.

5.2 OT-OCN 구간 침해를 가정한 위협 모델과 대응 설계 (Assume Breach)

본 시스템은 "어떤 구간도 절대적으로 안전하다"는 가정 위에 서 있지 않다. 오히려 현실적인 보안 설계 원칙에 따라, 내부 통신 구간과 전용 실행 채널까지 포함해 **침해 가능성을 항상 전제**로 한다. 따라서 본 절에서는 서버와 HSM 사이의 OT-OCN 스타일 내부 실행 채널이 침해되었거나, 공격자가 해당 채널의 트래픽을 관찰·조작할 수 있는 상태를 가정한다. 이 가정 하에서 핵심 질문은 하나다. **"그럼에도 자산이 실제로 탈취될 수 있는가?"**

먼저 "OT-OCN 구간이 뚫린다"는 표현이 무엇을 의미하는지 범위를 명확히 해야 한다. 일반적으로 내부 채널이 침해되었다고 말할 때는 네 가지 상황 중 하나에 해당한다. 첫째는 트래픽 스니핑, 즉 패킷을 엿보는 상황이다. 둘째는 과거 메시지를 다시 보내는 재전송(Replay) 공격이다. 셋째는 공격자가 위조된 메시지를 채널에 주입하는 상황이다. 넷째는 OT-OCN 릴레이 장비나 노드 자체가 침해되어 패킷을 떨어뜨리거나 지연시키거나 순서를 바꾸는 등 네트워크 동작을 교란하는 상황이다. 본 시스템은 이 네 가지를 각각 분리하여, 공격자가 무엇을 할 수 있고 무엇을 할 수 없는지를 논리적으로 검증한다.

첫 번째로, OT-OCN 채널의 트래픽이 노출되어 공격자가 패킷을 관찰할 수 있는 상황을 보자. 이때 공격자가 얻을 수 있는 것은 내부 라우팅에 사용되는 Name 토큰과 암호화된 payload, 그리고 트래픽이 흐르는 타이밍 같은 메타데이터다. 그러나 공격자가 얻지 못하는 것이 훨씬 더 중요하다. 공격자는 실제 자산 이동에 필요한 핵심 의미—예를 들어 대상 주소, 금액, 체인 정보, 사용자 키, 그리고 정책 조건—을 알 수 없다. 이유는 단순하다. 내부 Name은 설계상 의미가 제거된 라우팅 토큰이며, 실제 의미가 담긴 데이터는 종단간 암호화된 payload에만 존재하고, 그 의미 해석은 오직 HSM 내부에서만 수행되도록 되어 있기 때문이다. 따라서 트래픽이 노출되더라도 공격자는 "무슨 거래가 진행되는지"를 이해하거나 "자산 이동을 유발하는 실행 트리거"를 만들어낼 수 없다. 다만 트래픽의 양이나 시간 패턴 같은 운영적 메타데이터는 관찰될 수 있으므로, 이는 보안 침해라기보다 **운영 관점의 프라이버시/가용성 이슈**로 관리해야 한다.

두 번째로, 공격자가 과거의 정상 메시지를 그대로 다시 보내는 재전송(Replay) 공격을 생각해보자. 이런 공격은 "유효했던 요청을 반복 실행시켜 이중 출금이나 중복 서명을 유도하는 방식"으로 나타난다. 본 시스템은 이를 구조적으로 막기 위해 요청마다 nonce와 TTL(유효시간)을 포함하고, 요청 컨텍스트를 요약한 ctx_hash의 단일성을 보장하며, HSM 내부에서 이미 처리된 요청을 기억하는 replay cache를 운영한다. 이 설계는 중요한 의미를 갖는다. 설령 공격자가 과거의 "정상적으로 보이는 패킷"을 그대로 가지고 있다고 해도, 동일 요청은 다시 유효해질 수 없고, HSM은 이를 재실행하지 않는다. 즉, OT-OCN 채널이 어느 정도 침해되더라도 **한 번 실행된 요청이 다시 실행되는 상황은 발생하지 않도록 설계된다.**

세 번째로, 공격자가 위조된 메시지를 채널에 주입하는 상황을 보자. 이 시나리오에서 공격자는 임의의 Name을 만들어 보거나, payload를 조작해 HSM이 실행하도록 유도하려 할 수 있다. 그러나 본 시스템에서 HSM은 단순히 "채널로 들어왔다"는 이유만으로 어떤 요청도 실행하지 않는다. HSM은 먼저 server_sig와 user_cold_sig를 검증하고, Name과 payload가 의도한 방식으로 바인딩되어 있는지 확인하며, 마지막으로 정책 엔진 평가를 통해 금액 한도, 주소 통제, 쿨다운, 다중 승인 요구 등 조건을 통과하는지를 확인한다. 이 검증 체인을 통과하지 못하면 요청은 즉시 폐기된다. 따라서 공격자가 OT-OCN 채널을 통해 무언가를 주입할 수 있다고 해도, 그것이 **실행 권한으로 연결되지 않는다**. 여기서 핵심은 "채널은 열려 있을 수 있지만, 실행 권한은 열려 있지 않다"는 구조적 분리다.

네 번째로, OT-OCN 노드나 릴레이 장비 자체가 침해된 상황을 보자. 이 경우 공격자는 트래픽을 떨어뜨리거나, 지연시키거나, 순서를 변경하거나, 노이즈를 주입하는 방식으로 네트워크 동작을 교란할 수 있다. 즉, 가용성을 떨어뜨리는 행위는 가능하다. 그러나 이 공격자가 할 수 없는 일이 분명히 존재한다. 공격자는 유효한 사용자 서명이나 서버 서명을 생성할 수 없고, HSM 정책을 우회할 수 없으며, 그 결과 자산 이동을 실행할 수 없다. 따라서 이 시나리오는 보안의 관점에서 "자산 탈취"가 아니라 "서비스 장애(DoS)"로 분류된다. 즉, OT-OCN 구간이 침해되더라도 **가용성 리스크는 생길 수 있으나, 무결성과 자산 안전성은 유지되도록 설계되어 있다.**

이제 핵심 질문으로 돌아가자. "그래서 OT-OCN 구간이 단독으로 침해되면 자산이 실제로 털릴 수 있는가?" 답은 명확하다. **OT-OCN 구간만 침해되어서는 자산 이동이 발생하지 않는다.** 그 이유는 시스템의 실행 조건이 단 하나가 아니라, 여러 조건을 동시에 충족해야만 성립하도록 설계되어 있기 때문이다. 자산 이동이 실행되려면 최소한 사용자 개인 서명(user_cold_sig), 서버 서명(server_sig), Name–payload 바인딩의 정합성, HSM 정책 통과, 그리고 중복·재생 방지 조건이 동시에 충족되어야 한다. OT-OCN은 이 조건들 중 어느 하나도 단독으로 만들어내지 못하며, 따라서 채널이 뚫렸다는 사실만으로 실행이 일어나지 않는다.

이 설계가 의미하는 바는 분명하다. 본 시스템에서 OT-OCN은 "절대적으로 신뢰해야 하는 구간"이 아니다. 오히려 **침해를 가정해도 되는 구간**으로 설계되었다. 즉, OT-OCN 자체가 완벽하다고 주장하지 않으며, OT-OCN이 깨져도 최종 안전성이 유지되도록 구조를 짰다. 이것이 바로 제도권 금융이나 국가 단위 인프라에서 채택하는 "Assume Breach(침해 가정)" 사고 방식이다.

이 관점은 앞서 논의했던 개인 지갑(예: Ballet, 키페어) 사고와도 직접적으로 연결된다. 개인 키가 유출되거나 생성 과정에 문제가 생겨 공격자가 키를 알게 되는 상황은 현실적으로 완전히 배제할 수 없다. 그러나 본 시스템에서는 키가 유출되더라도, 실행 경로와 정책이 분리되어 있기 때문에 **전액을 즉시 탈취하는 형태의 사고가 구조적으로 어려워진다.** 기존 개인 지갑 사고가 "키를 알면 곧바로 실행 가능"이라는 단일 실패 구조에서 발생했다면, 본 시스템은 "키를 알아도 실행이 되지 않는 조건 누적 구조"를 통해 손실의 형태 자체를 바꾼다.

정리하면, OT-OCN 구간이 침해되었을 때 발생할 수 있는 문제는 주로 가용성 측면의 리스크(지연, 드롭, 서비스 장애)이며, 자산이 즉시 탈취되는 형태의 사고로 곧바로 연결되지는 않는다. 그 이유는 단 하나다. **OT-OCN은 실행 권한을 갖지 않기 때문**이다. 본 시스템은 OT-OCN을 보안의 근거로 삼지 않으며, 오히려 침해 가능성을 전제로 하되, 최종 실행 권한은 개인 승인과 HSM 정책 및 최종 집행 분리 구조에 의해 보호되도록 설계한다.

마지막으로 이 절을 한 문장으로 요약하면 다음과 같다.

"본 시스템은 OT-OCN 구간의 침해를 가정하더라도 자산 이동이 발생하지 않도록 설계된 구조적 보안 아키텍처이다."

5.3 요약: 위협을 '막는' 것이 아니라 '무력화하는 구조'

본 장에서 설명한 위협 모델의 핵심은, 특정 공격 기법을 완벽히 차단하는 것이 아니라 **어떤 구간이 침해되더라도 자산 이동으로 이어지지 않도록 설계**하는 것이다. 서버, 사용자 키, 내부 통신 채널 중 어느 하나가 침해되더라도, 단일 요소만으로는 자산 이동이 실행되지 않는다. 이는 디지털 자산이 제도권 금융 인프라로 편입되기 위해 요구되는 **책임 분리, 승인 증명, 정책 강제**라는 요건을 기술적으로 충족하는 방식이다.

6. 보안 강점 요약

본 시스템의 보안 강점은 특정 구성 요소의 성능이나 단일 기술에 의존하지 않는다. 대신, 자산 이동과 관련된 권한과 책임을 구조적으로 분리함으로써, 하나의 실패나 침해가 곧바로 자산 손실로 이어지지 않도록 설계되었다.

우선, 서버가 침해되더라도 단독으로 자산을 이동시킬 수 없다. 서버는 요청을 전달하는 역할만 수행하며, 사용자의 개인 콜드월렛에서 생성된 승인 서명이 없는 요청은 실행될 수 없다. 이는 서버 침해 사고가 즉각적인 자산 손실로 이어지는 기존 구조와 근본적으로 다르다.

또한 사용자 개인 키만으로도 자산 이동은 불가능하다. 설령 사용자의 키가 탈취되는 상황이 발생하더라도, HSM은 사전에 정의된 정책 엔진을 통해 금액 한도, 주소 관리, 승인 절차 등을 검증한다. 정책 조건을 충족하지 못하는 요청은 자동으로 차단되거나 보류된다.

한편, HSM 역시 단독으로 정책을 우회할 수 없다. HSM은 최종 집행자이지만, 사용자 승인과 서버를 통한 정상 요청 흐름이 함께 충족되지 않으면 실행을 수행하지 않는다. 이는 내부자나 시스템 오작동에 의한 단독 실행 가능성을 구조적으로 제거한다.

결과적으로 **본 시스템에서 자산 이동은 사용자 개인 콜드월렛, 서버, 그리고 HSM이라는 세 주체의 조건이 동시에 충족될 때만 가능하다.** 이 다중 조건 구조는 단일 침해나 단일 오류로는 자산이 이동할 수 없도록 하여, 수탁 환경에서 요구되는 높은 수준의 안전성과 신뢰성을 제공한다.

7. 운영 및 컴플라이언스

본 시스템은 기술적 보안뿐 아니라, 실제 수탁 사업 운영과 규제 환경을 고려한 구조로 설계되었다. 디지털 자산 수탁에서 중요한 것은 사고를 막는 것뿐만 아니라, 사고가 발생했을 때 **무엇이 어떻게 승인되었는지를 명확히 설명할 수 있는가**이다.

우선, 본 구조에서는 모든 자산 이동 요청에 대해 명확한 서명 증거가 존재한다. 사용자의 개인 콜드월렛 서명, 서버의 중계 서명, 그리고 HSM의 최종 실행 서명이 각각 분리되어 기록되며, 이는 각 단계에서 누가 어떤 역할을 수행했는지를 명확히 증명한다. 이를 통해 승인 과정 전체가 사후에 재현 가능하다.

또한 자산 이동은 단순히 "허용되었다"는 결과만 남기지 않는다. HSM 내부의 정책 엔진은 어떤 정책 룰이 적용되어 해당 요청이 승인되었는지를 함께 기록하며, 각 승인 건은 특정 정책 룰 ID와 연결된다. 이는 감사 시점에서 "왜 이 거래가 허용되었는가"를 설명할 수 있는 근거가 된다.

주소 관리 역시 체계적으로 이루어진다. 새로운 주소의 등록, 변경, 삭제 이력은 모두 기록되며, 각 변경은 정책에 따라 쿨다운 기간이나 추가 승인 절차를 요구할 수 있다. 이를 통해 주소 변경을 통한 우회 출금과 같은 사고를 예방할 수 있다.

본 시스템은 또한 외부의 AML 및 리스크 평가 시스템과의 연계를 전제로 설계되었다. 거래 요청은 리스크 스코어링 결과에 따라 자동으로 보류되거나 추가 승인을 요구할 수 있으며, 이는 자금세탁 방지 및 이상 거래 탐지 체계와 자연스럽게 연결된다.

이와 같은 구조는 단순한 기술적 보안을 넘어, 금융기관이 요구하는 수탁 요건 "명확한 승인 절차, 감사 가능성, 책임 분리, 정책 기반 통제"을 충족할 수 있는 기반을 제공한다. 따라서 본 시스템은 규제 환경에서도 운영 가능한 실질적인 디지털 자산 수탁 아키텍처를 목표로 한다.

8. 시장 기회

본 시스템이 목표로 하는 시장은 단일한 고객군에 국한되지 않는다. 디지털 자산을 **관리·보관·운용**해야 하는 다양한 주체들이 공통적으로 직면한 문제를 해결하는 것을 목표로 한다. 이들 주체의 공통된 요구는 단순히 자산을 안전하게 보관하는 것을 넘어, **자산 이동에 대한 승인 책임을 명확히 하고, 정책에 따라 통제할 수 있는 구조**를 갖추는 것이다.

특히 최근 한국을 포함한 주요 국가에서 **STO(증권형 토큰) 시장의 법제화가 본격적으로 논의·추진**되면서, 디지털 자산 관리에 대한 요구는 한 단계 더 구체적이고 엄격해지고 있다. STO는 기존 암호자산과 달리 증권적 성격을 가지며, 발행·유통·보관 전 과정에서 명확한 책임 구조와 감사 가능성이 요구된다. 이는 단순한 지갑 보관이나 서버 기반 수탁 구조로는 충족하기 어려운 영역이다.

첫째, **기관 수탁사**는 이러한 변화의 가장 직접적인 수혜자이자 핵심 대상이다. 기관 수탁사는 대규모 자산을 관리하면서도 금융 규제, 내부 통제, 외부 감사 요구를 동시에 충족해야 한다. 사용자 승인 기반의 자산 이동과 HSM 중심의 최종 집행 구조는, STO를 포함한 규제 자산 환경에서 요구되는 보안 수준과 운영 투명성을 동시에 제공할 수 있는 현실적인 대안이 된다.

둘째, **거래소** 역시 중요한 시장이다. 거래소는 상시적으로 높은 트래픽과 빈번한 자산 이동을 처리해야 하며, 서버 침해 사고가 곧 대규모 자산 손실로 이어질 수 있는 구조적 위험을 안고 있다. 여기에 더해, 향후 STO와 스테이블코인이 본격적으로 거래소 인프라에 편입될 경우, 단순한 출금 통제를 넘어 정책 기반 승인과 책임 분리가 필수 요건이 된다. 본 시스템은 서버 단독 침해로는 자산이 이동되지 않는 구조를 제공함으로써, 거래소의 운영 리스크를 구조적으로 낮출 수 있다.

셋째, **OTC 데스크**는 고액 거래를 다루는 특성상 승인 절차와 책임 분리가 특히 중요한 영역이다. 특히 기관 간 STO 거래나 대규모 스테이블코인 결제·정산이 늘어날수록, 거래 승인에 대한 명확한 증거와 정책 집행이 요구된다. 본 시스템은 다중 승인, 금액 한도, 주소 관리 정책을 구조적으로 적용할 수 있어, 고액 거래 환경에서도 통제 가능한 자산 이동을 가능하게 한다.

넷째, **Web3 기업의 재무(Treasury) 관리** 영역에서도 본 시스템은 높은 활용 가치를 가진다. Web3 기업은 탈중앙화 환경에서 운영되면서도, 실제 사업 운영에서는 내부 통제, 승인 절차, 감사 요구를 피할 수 없다. 특히 스테이블코인을 기반으로 한 결제, 급여, 정산 구조가 확대될수록, 자산 이동에 대한 정책적 통제와 책임 분리가 필수적이다. 개인 승인과 기업 정책을 결합한 본 구조는 이러한 요구를 동시에 만족시킨다.

마지막으로, **고액자산가 및 패밀리 오피스**와 같은 개인 고객층 역시 중요한 시장으로 부상하고 있다. 이들은 자산의 실질적인 통제권은 유지하면서도, 사고 방지, 상속, 장기 관리 측면에서 제도적 보호와 구조화된 통제를 원한다. 특히 STO나 스테이블코인 기반 자산이 늘어날수록, 단순 개인 지갑을 넘어선 관리 체계가 필요해진다. 개인 콜드월렛 승인과 수탁 HSM 정책을 결합한 본 구조는 이러한 요구에 부합하는 현실적인 대안을 제시한다.

이처럼 본 시스템은 **한국을 포함한 STO 시장의 제도화 흐름과 스테이블코인 기반 금융 인프라의 확산**이라는 거대한 변화 속에서, 기관과 개인, 중앙화와 분산화의 경계에 위치한 다양한 고객군을 포괄한다. 이는 단기적인 기술 트렌드를 넘어, 디지털 자산 관리 시장 전반에 적용 가능한 장기적 구조를 제시한다.

9. 비즈니스 모델

본 시스템의 비즈니스 모델은 단순한 소프트웨어 라이선스 판매가 아니라, **디지털 자산의 보관·이동·통제 과정 전반에서 발생하는 가치에 기반한 구조**를 지향한다. 이는 특히 한국 시장에서 진행 중인 STO 법제화와 스테이블코인 활용 확대 흐름과 직접적으로 맞닿아 있다.

9.1 Custody AUM Fee (자산 보관 수수료)

가장 기본이 되는 수익 모델은 **수탁 자산 규모(AUM)에 기반한 보관 수수료**이다. 기관 수탁사, 거래소, STO 발행·유통 사업자는 일정 규모 이상의 디지털 자산을 장기간 보관해야 하며, 이 과정에서 단순 저장이 아닌 **정책 기반 통제와 감사 가능한 구조**를 요구한다.

특히 STO 자산의 경우, 증권적 성격을 가지는 만큼 "누가 언제 어떤 조건으로 자산 이동을 승인했는지"에 대한 기록과 통제가 필수적이다. 본 시스템은 이러한 요구를 충족하는 수탁 인프라로서, 보관 자산 규모에 비례한 안정적인 반복 수익 구조를 형성할 수 있다.

이는 국내 금융기관과 수탁 사업자가 선호하는 **예측 가능한 수익 모델**이기도 하다.

9.2 출금·서명 트랜잭션 Fee

두 번째 수익원은 **출금 또는 서명 실행 단위로 부과되는 트랜잭션 수수료**이다. 본 시스템에서는 모든 자산 이동이 HSM의 최종 승인과 서명을 거쳐야 하며, 이 과정 자체가 하나의 고부가가치 서비스로 작동한다.

특히 다음과 같은 경우에서 활용도가 높다.

- 기관 자산 출금
- OTC 고액 거래 승인
- STO 이전·정산
- 스테이블코인 기반 대량 결제·정산

한국 시장에서는 거래 건수보다 **"한 건의 책임과 위험"이 큰 경우**가 많기 때문에, 건별 서명·승인에 대한 과금 모델은 충분한 수용 가능성을 가진다.

.

9.3 정책 엔진 라이선스

세 번째 수익 모델은 **정책 엔진 자체에 대한 라이선스 제공**이다. 모든 고객이 동일한 정책을 사용하는 것은 아니며, 특히 기관 고객과 STO 사업자는 각기 다른 내부 통제 기준과 규제 요건을 가진다.

본 시스템의 정책 엔진은 다음과 같은 요소를 구조적으로 정의할 수 있다.

- 금액 한도
- 주소 화이트리스트/블랙리스트
- 신규 주소 쿨다운
- 다중 승인(M-of-N)
- 특정 시간대·조건 제한

국내 금융기관 및 STO 관련 사업자에게는 이 정책 엔진을 **자사 내부 통제 시스템과 연동하거나 커스터마이즈**할 수 있는 권한 자체가 중요한 가치가 된다. 따라서 정책 엔진은 단순 기능이 아니라 **엔터프라이즈 라이선스 모델**로 제공될 수 있다.

9.4 감사·컴플라이언스 모듈 과금

마지막으로 중요한 수익원은 **감사 및 컴플라이언스 모듈**이다. 한국 시장에서는 기술적 보안만큼이나, 금융감독·내부 감사·외부 회계법인 대응이 실제 사업 운영의 핵심 요소로 작용한다.

본 시스템은 다음과 같은 데이터를 구조적으로 제공한다.

- 사용자 승인 서명 증거
- 서버 중계 이력
- HSM 정책 적용 결과
- 정책 룰 ID 기반 승인 사유

이러한 정보는 단순 로그가 아니라, "설명 가능한 승인 기록"으로 정리될 수 있으며, 이는 STO 발행사, 수탁사, 거래소가 규제 대응과 내부 통제를 수행하는 데 직접적인 가치를 제공한다.

감사 리포트 자동 생성, 규제 대응용 데이터 패키징, 외부 감사 대응을 위한 전용 인터페이스 등은 별도의 **컴플라이언스 모듈 과금**으로 이어질 수 있다.

9.5 한국 시장에서의 종합적 의미

한국은 현재 STO 제도화와 스테이블코인 활용 논의가 동시에 진행되고 있는 시장이다. 이는 곧 디지털 자산이 "실험적 기술"에서 **제도권 금융 자산 관리의 영역**으로 이동하고 있음을 의미한다.

이러한 환경에서 본 시스템의 비즈니스 모델은 다음과 같은 특징을 가진다.

- 단기 유행이 아닌 **장기 운영형 수익 구조**
- 규제 강화와 함께 **가치가 커지는 모델**
- 기관·기업·고액자산가를 동시에 포괄하는 확장성

즉, 본 시스템은 기술을 파는 것이 아니라, **디지털 자산을 제도권에서 관리할 수 있게 해주는 인프라를 제공하는 사업 모델**을 지향한다.

10. 로드맵

본 시스템의 로드맵은 기술 발전만을 기준으로 하지 않는다. 한국을 포함한 주요 시장에서 진행 중인 **STO 제도화와 스테이블코인 도입 흐름**을 전제로, 제도 수용 가능성과 실제 사업 적용 가능성을 함께 고려하여 단계적으로 설계되었다.

Phase 1: 사용자 승인 + HSM 최종 집행 구조 확립

첫 번째 단계의 목표는 단순하지만 가장 중요하다. **사용자 개인 승인(user_cold_sig)과 HSM 최종 집행 구조를 완성하는 것**이다. 이 단계에서는 모든 자산 이동이 반드시 사용자의 개인 콜드월렛 서명을 포함하도록 하고, HSM이 이를 검증한 후에만 실행되도록 한다. 이를 통해 서버 침해나 내부 시스템 오류가 곧바로 자산 손실로 이어지는 기존 구조를 근본적으로 차단한다. Phase 1은 현재 한국 시장에서 논의 중인 **디지털 자산 수탁 가이드라인 및 최소 보안 요건**을 충족하는 단계이며, 기존 암호자산 수탁 및 초기 STO 파일럿 환경에 즉시 적용 가능하다.

Phase 2: 정책 엔진 고도화 및 주소 관리 체계 구축

두 번째 단계에서는 자산 이동에 대한 **정책 통제 능력**을 본격적으로 강화한다.

이 단계에서 시스템은 다음과 같은 기능을 제공한다.

- 금액 한도 정책
- 주소 화이트리스트 및 블랙리스트
- 신규 주소 등록 시 쿨다운 기간 적용

이는 단순한 기술적 기능이 아니라, 금융기관과 STO 사업자가 요구하는 내부 통제 요건을 시스템 차원에서 충족하기 위한 핵심 요소이다. Phase 2는 한국의 STO 제도화가 구체화되며 "누가 어떤 조건으로 증권형 토큰을 이전할 수 있는가"가 실제 규정으로 명시되기 시작하는 시점과 자연스럽게 맞물린다.

Phase 3: VC/VP 통합 및 감사 자동화

세 번째 단계에서는 신원·자격 기반 통제와 감사 자동화가 결합된다.

이 단계에서 시스템은:

- VC/VP 기반 사용자·조직 자격 검증
- 승인 과정 전반에 대한 자동 감사 로그 생성
- 정책 룰 ID 기반 승인 사유 기록

을 제공한다.

이는 STO 발행사, 수탁사, 거래소가 외부 감사, 내부 통제, 규제 대응을 수행하는 데 직접적인 가치를 제공한다. Phase 3는 스테이블코인 기반 결제·정산이 확대되고, "설명 가능한 승인 기록"이 요구되는 시점에 핵심 경쟁력이 된다.

Phase 4: 기관 연계 API 및 다체인 확장

마지막 단계에서는 본 시스템을 **기관 간 디지털 자산 인프라의 일부**로 확장한다.

- 금융기관·수탁사·거래소 연계 API
- 다양한 블록체인 및 스테이블코인 네트워크 지원
- STO 유통·정산 시스템과의 연동

이를 통해 본 시스템은 단일 서비스가 아니라, **제도권 디지털 자산 시장의 실행 레이어**로 자리 잡는 것을 목표로 한다.

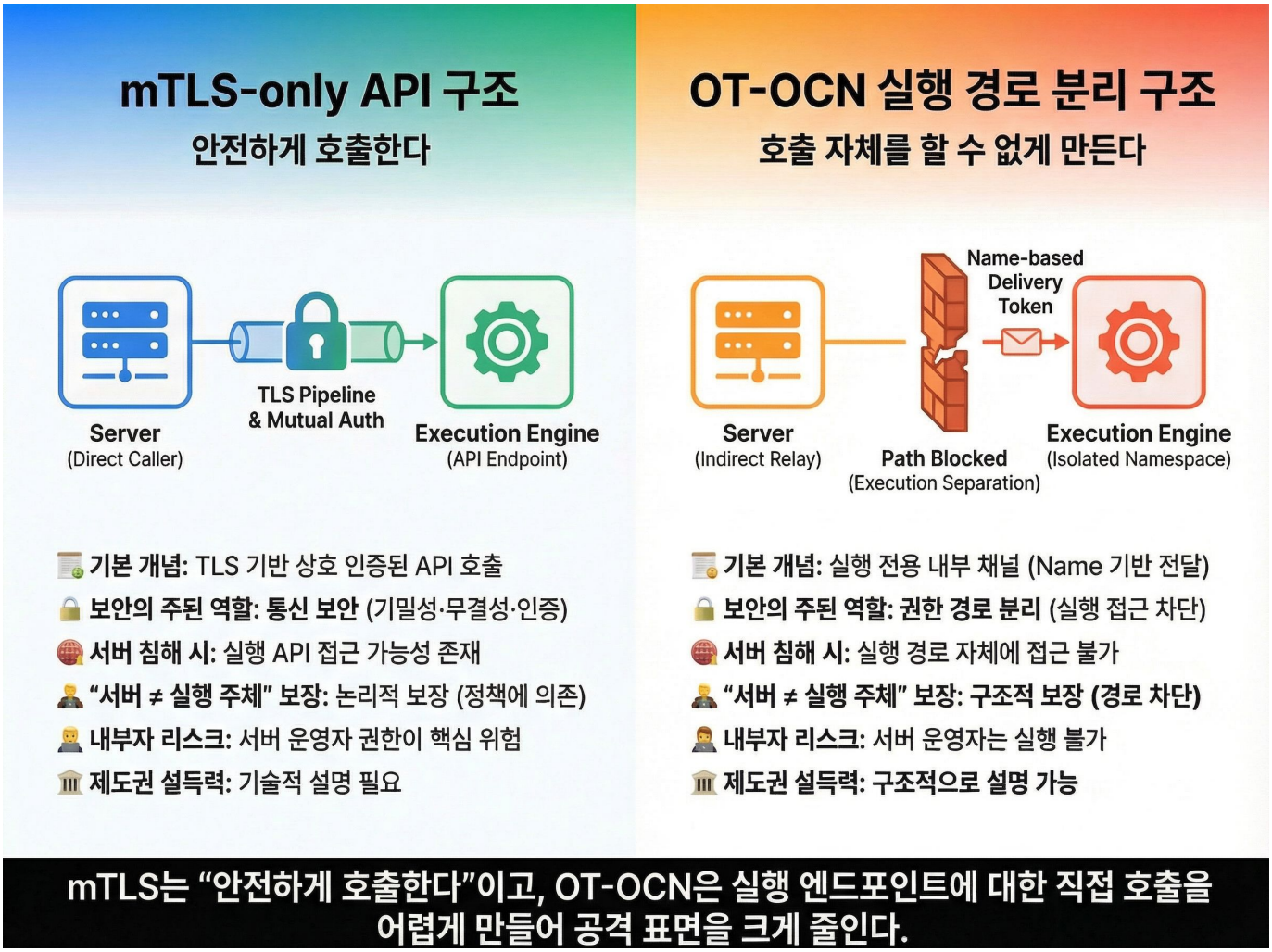
요약

본 백서가 가장 분명하게 강조하고자 하는 점은 다음과 같다.

본 시스템의 보안은 특정 네트워크 기술이나 단일 장비에 의존하지 않는다. 보안의 근거는 개인 서명에 의한 명시적 승인, HSM 내부 정책에 따른 강제, 그리고 최종 집행 권한의 분리라는 구조적 원칙에 있다.

이 구조가 실제 운영 환경에서도 훼손되지 않도록 하기 위해, 서버에서 HSM으로 이어지는 실행 경로는 일반적인 API 호출 경로와 **구조적으로 분리된 OT-OCN 스타일의 전용 전달 채널**을 사용한다. OT-OCN은 보안을 대신하는 기술이 아니라, **서버가 실행 권한에 접근하거나 우회하지 못하도록 하는 필수적인 실행 경로 분리 장치**이며, 이 채널 없이는 본 시스템이 의도한 권한 분리 구조는 완전하게 성립할 수 없다.

본 시스템은 새로운 네트워크를 주장하지 않는다. 대신, 디지털 자산이 제도권 금융으로 편입되는 과정에서 반드시 요구되는 질문, 즉 누가 승인하고, 누가 정책을 집행하며, 누가 최종 실행하는가를 **OT-OCN을 포함한 구조적 분리**를 통해 기술적으로 명확히 구분하고, 증명 가능한 형태로 제시한다.



왜 이 차이가 STO·수탁·스테이블코인에서 중요한가

위의 인포그래픽은 mTLS-only API 구조와 OT-OCN 기반 실행 경로 분리 구조의 차이를 시각적으로 비교한다. 두 구조 모두 보안을 목표로 하지만, 보안이 성립되는 지점과 그 방식은 근본적으로 다르다.

mTLS-only 구조의 한계

mTLS-only 구조에서 서버는 상호 인증된 TLS 채널을 통해 실행 엔진(HSM 또는 실행 시스템)을 직접 호출한다. 이 방식은 통신 구간의 기밀성·무결성·인증을 보장하는 데에는 효과적이다. 그러나 구조적으로 보면 서버는 여전히 실행 API의 위치와 형태를 알고 있으며, 실행 요청이 무엇을 의미하는지 이해한다.

즉, 서버는 "조건만 충족되면 실행을 트리거할 수 있는 위치"에 놓여 있다. 이때 조건이란 정책, 코드, 권한 설정에 의해 통제되며, 이는 본질적으로 논리적 보장에 해당한다. 서버가 침해되거나, 내부 권한이 오남용되거나, 설정이 잘못된 경우 실행 경로는 그대로 노출된다.

이러한 구조는 규제 및 감독 관점에서 **단일 실패 지점(Single Point of Failure, SPOF)**으로 해석될 수 있다. "서버가 왜 실행을 호출했는가"를 사후에 설명해야 하며, 구조 자체로 위험을 제거했다고 보기 어렵다.

OT-OCN 실행 경로 분리 구조의 효과

OT-OCN 기반 구조에서는 서버의 역할이 근본적으로 달라진다. 서버는 실행 API를 호출하지 않으며, 실행 엔진이 위치한 경로 자체에 접근할 수 없다. 대신 서버는 의미가 제거된 Name 기반 전달 토큰만을 내부 전용 채널을 통해 전달한다.

실행 경로는 일반적인 API 모델과 분리된 독립된 네임스페이스로 구성되며, 이 경로는 서버가 직접 호출하거나 우회할 수 없다. 실제 실행 여부는 오직 HSM 내부에서, 자기 정책 평가와 서명 검증 결과에 따라 결정된다.

이로 인해 "서버 침해 = 자산 침해"라는 기존의 위험 공식은 구조적으로 성립하지 않게 된다. 서버는 승인 주체도, 집행 주체도 아니며, 실행 경로에 대한 접근 권한 자체를 가지지 않는다. 이는 보안을 기술이나 설정이 아니라 구조로 설명할 수 있게 만드는 결정적 차이다.

감독기관·금융권 설명을 위한 핵심 메시지

이 차이는 기술적인 미세 조정이 아니라, 제도권 금융이 요구하는 안전성의 수준을 충족하는지 여부를 가르는 기준이다. 다음 문장들은 감독기관, 금융기관, 감사 조직에 그대로 사용할 수 있는 설명이다.

- "mTLS는 통신을 안전하게 만드는 기술이고, 저희는 집행 권한의 경로를 구조적으로 분리합니다."
- "서버는 실행 API를 원천적으로 호출할 수 없습니다."
- "실행 경로는 네트워크 설정이 아니라 아키텍처 구조로 보호됩니다."
- "따라서 서버 침해는 승인 침해나 자산 침해로 이어지지 않습니다."

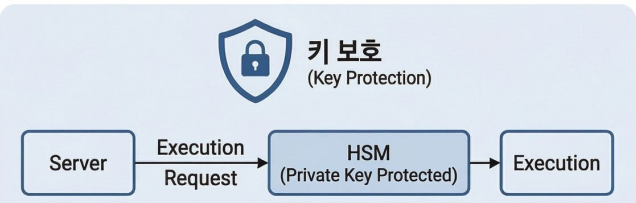
이러한 설명이 가능하다는 점 자체가, OT-OCN 기반 실행 경로 분리 구조가 STO, 수탁, 스테이블코인과 같은 제도권 자산 관리 환경에 적합한 이유이다.

부록 B (Comparison Chart of OT-OCN & PKCS #11 + HSM)

구분	PKCS #11 + HSM 구조	OT-OCN 기반 실행 경로 분리 구조
기본 목적	Private key 보호	Execution authority 분리 및 보호
핵심 보호 대상	Key material	Execution path + execution authority
Private key 저장	HSM 내부 저장 (외부 노출 없음)	HSM 내부 저장 (동일)
Execution 요청 방식	Server가 PKCS #11 API 호출	Server는 opaque 전달 토큰만 전달
Execution endpoint addressability	Server가 endpoint 인지 가능	Execution endpoint 직접 address 불가
Execution trigger 위치	Server infrastructure에서 시작	Execution core 내부에서만 최종 결정
Server compromise 시 영향	Execution 요청 생성 가능	Execution authority 직접 행사 불가
Network layer 의존성	IP 기반 API 호출 구조	Name-based opaque execution channel
Execution path exposure	Addressable execution path 존재	Non-addressable execution path 구조
Policy enforcement 위치	Application + HSM	Execution core (HSM 내부 정책 중심)
Replay / injection 공격 surface	API endpoint 존재	Opaque execution channel로 축소
Execution authority isolation	부분적	구조적으로 분리
Attack surface 범위	Application server + API layer	Execution core 중심으로 제한
Execution integrity 보장 수준	Infrastructure dependent	Architecture-level enforced
금융기관 수탁 구조 적합성	Standard 수준	Institutional-grade execution isolation
Zero-trust / Assume Breach 모델 적합성	제한적	구조적으로 적합
감사 및 책임 분리 명확성	논리적 분리	구조적 분리

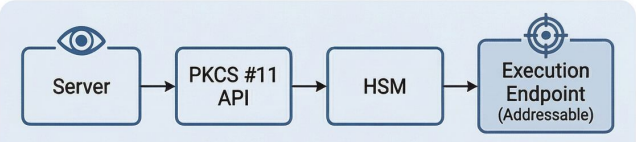
PKCS #11 + HSM 구조

1 핵심 차이: 보호 대상



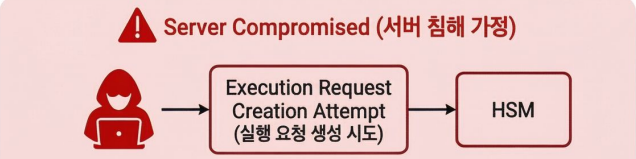
- Private key는 보호됨
- 그러나 execution request는 여전히 server에서 시작됨

2 Execution Endpoint 주소 지정 가능성



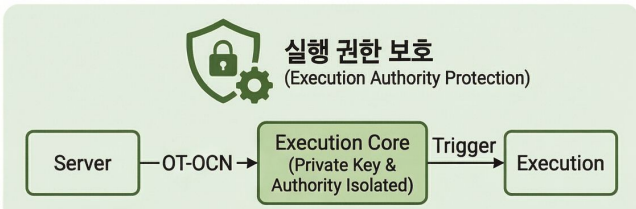
- Server는 execution endpoint 존재 인지
- Server는 execution request 생성 가능

3 Assume Breach 상황에서의 구조적 차이

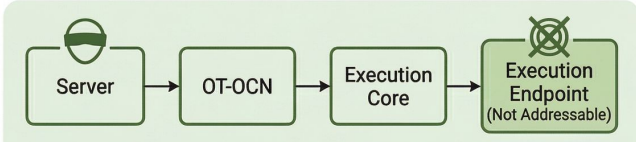


- Attacker가 execution request 생성 시도 가능

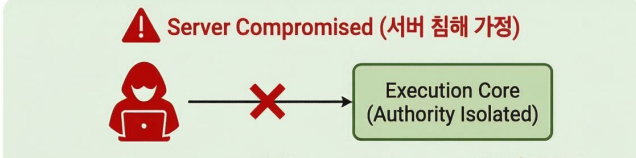
OT-OCN 구조



- Private key 보호 + execution authority isolation
- Execution trigger가 execution core 내부에서만 최종 결정됨



- Server는 execution endpoint 직접 address 불가
- Server는 execution authority 없음



- Execution authority 자체가 execution core 내부에 격리됨
- Execution trigger 권한 없음

부록 C (Q & A)

Q1. 서버가 침해되면 자산이 이동할 수 있지 않습니까?
A. 서버는 실행 경로에 접근할 수 없으며, 사용자 서명과 HSM 정책이 없으면 실행은 불가능합니다.

Q2. 서버가 실행 API를 호출할 수 있습니까?
A. 아닙니다. 실행 API는 서버가 접근할 수 없는 전용 경로에 존재합니다.

Q3. 통신 보안은 어떻게 보장합니까?
A. 통신은 mTLS 및 메시지 서명으로 보호되며, 실행 권한 보호는 구조적으로 분리됩니다.

Q4. 왜 일반 API 구조로는 부족합니까?
A. 일반 API 구조에서는 서버가 실행 호출 주체가 되며, 이는 단일 실패 지점이 됩니다.

Q5. OT-OCN은 새로운 네트워크입니까?
A. 아닙니다. 내부 실행 경로를 분리하기 위한 전달 방식입니다.

Q6. OT-OCN이 없으면 이 구조는 성립합니까?
A. 실행 권한 분리를 구조적으로 보장할 수 없습니다.

Q7. 내부자가 서버 권한을 악용할 수 있지 않습니까?
A. 서버 운영자는 실행을 수행할 수 없습니다.

Q8. 사용자 키가 탈취되면 어떻게 됩니까?
A. HSM 정책에 의해 금액·주소·쿨다운 제한이 적용됩니다.

Q9. 승인 기록은 남습니까?
A. 모든 승인에는 서명 증거와 정책 룰 ID가 기록됩니다.

Q10. 주소 변경은 어떻게 관리합니까?
A. 주소 변경 이력은 기록되며 정책에 따라 제한됩니다.

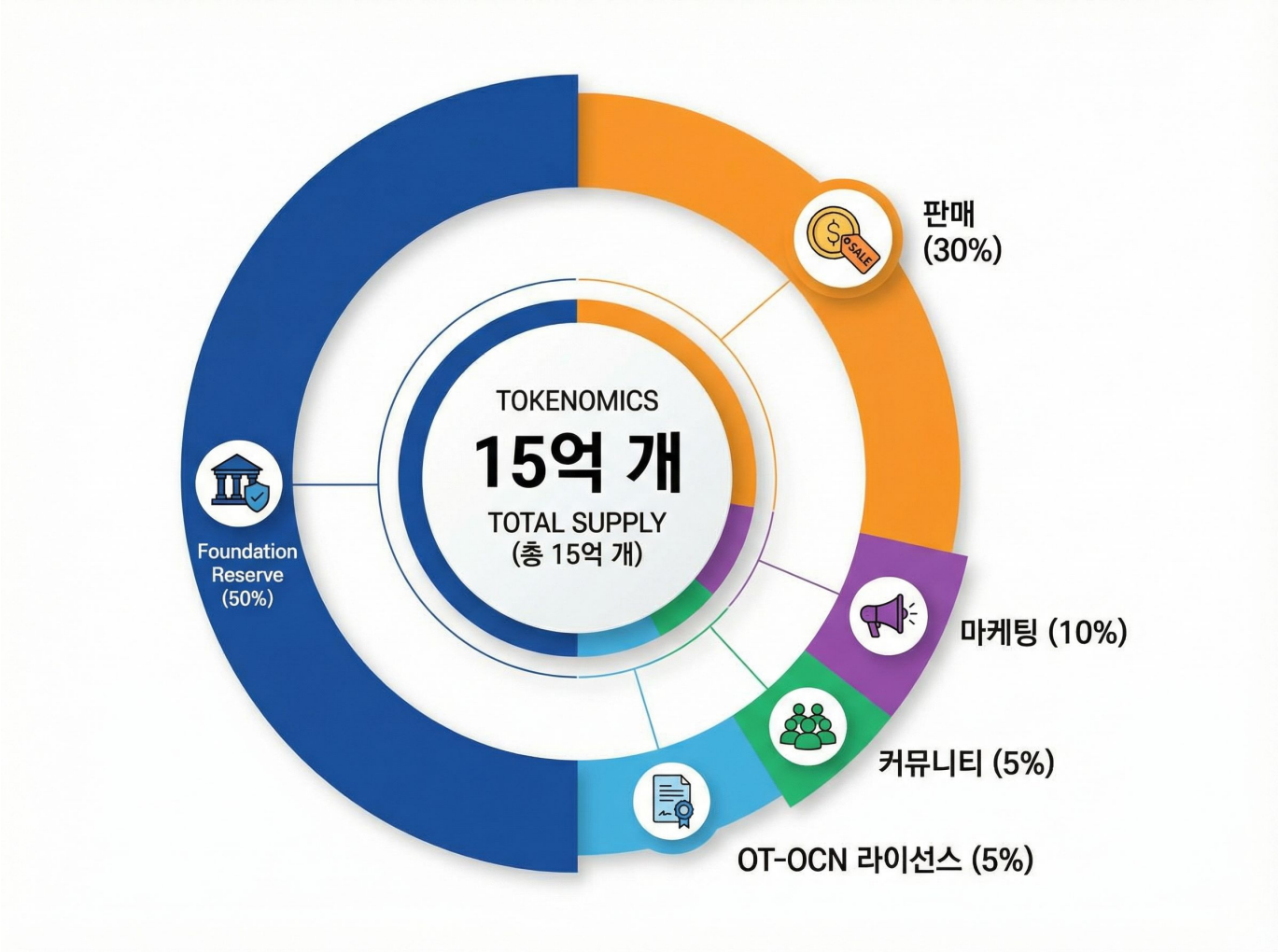
Q11. 감사 대응이 가능합니까?
A. 승인·정책·집행 전 과정이 감사 가능합니다.

Q12. 스테이블코인에도 적용 가능합니까?
A. 가능합니다. 체인과 무관한 실행 구조입니다.

Q13. STO 자산에 적용 가능한 이유는 무엇입니까?
A. 승인 책임과 집행 권한을 명확히 분리하기 때문입니다.

Q14. HSM이 단독으로 자산을 이동할 수 있습니까?
A. 사용자 승인과 서버 중계가 없으면 실행되지 않습니다.

Q15. 이 구조의 핵심 한 문장은 무엇입니까?
A. 보안은 통신이 아니라, 누가 승인하고 누가 실행하는지의 구조입니다.



QCAT Token Overview

QCAT 토큰은 단순한 결제 수단이나 보조적인 유틸리티 토큰이 아니다. 본 시스템에서 QCAT은 신원(Identity), 실행(Execution), 접근(Access), 정산(Settlement)을 연결하는 핵심 요소로 설계되었다.

본 플랫폼의 모든 주요 기능은 QCAT을 기반으로 동작하며,
QCAT 없이 핵심 기능의 수행이 불가능하도록 설계되어 있다.

이를 통해 QCAT은 선택적 사용이 아닌, 시스템 운영을 위한 필수적인 구조적 토큰(Structural Token)의 역할을 수행한다.

Identity Layer: Trust-backed Global Identity

본 시스템은 글로벌 디지털 신원 체계를 제공하며, QCAT은 신원 생성 및 검증 과정에서 신뢰를 담보하는 역할을 수행한다.

Identity Creation

사용자가 글로벌 ID를 생성하기 위해서는 일정량의 QCAT을 스테이킹해야 한다. 이는 무분별한 ID 생성 및 스팸 행위를 방지하기 위한 경제적 장치다.

Identity Verification

신원 인증(KYC, Verifiable Credentials 등)에 참여하는 검증자는 QCAT을 스테이킹해야 하며, 잘못된 인증을 수행할 경우 해당 스테이킹은 슬래싱될 수 있다. 이를 통해 신원 검증은 단순한 데이터 확인이 아닌, 경제적 책임이 수반된 신뢰 시스템으로 작동한다.

Identity Lifecycle Management

신원 정보의 갱신, 권한 변경, 인증 갱신 등의 모든 과정에서 QCAT이 사용되며, 이는 신원 관리의 지속적인 무결성과 비용 구조를 유지한다.

AI Layer: Tokenized Intelligence & Execution Control

본 플랫폼은 AI 기반 분석 및 자동화 기능을 제공하며, QCAT은 AI 서비스의 접근 및 실행을 통제하는 역할을 수행한다.

AI Service Access

사용자가 AI 분석, 추천, 데이터 처리 기능을 요청할 경우 QCAT을 기반으로 한 사용료(pay-per-use)가 발생한다.

AI-driven Execution

AI가 제안한 트랜잭션이나 실행 요청이 실제로 수행되기 위해서는 추가적인 QCAT 검증 및 사용이 필요하다. 이를 통해 단순한 AI 추천과 실제 자산 이동 간의 경계를 분리하고, AI에 의한 실행 리스크를 통제할 수 있다.

Tiered Intelligence Access

고급 AI 기능 및 프리미엄 분석 서비스는 QCAT 스테이킹 수준에 따라 접근 권한이 차등적으로 제공된다.

Payment & Settlement Layer

QCAT은 플랫폼 내 결제 및 크로스체인 정산을 위한 핵심 매개체로 사용된다.

Transaction Routing

다양한 네트워크 간 거래 수행 시, QCAT은 최적의 경로 선택 및 라우팅 메커니즘에 사용된다.

Settlement Fee

모든 결제 및 정산 과정에서 QCAT이 수수료로 사용되며, 이는 네트워크 유지 및 참여자 보상 구조로 분배된다.

Liquidity Participation

유동성 공급자는 QCAT을 스테이킹하여 네트워크 내 결제 처리 및 정산 과정에 참여할 수 있으며, 이에 따른 보상을 획득한다.

Execution Layer: Policy-Enforced Transaction Control

본 시스템의 핵심은 실행(Execution)에 대한 통제 구조이며, QCAT은 실행 권한을 제한하고 검증하는 역할을 수행한다.

Execution Authorization

모든 트랜잭션 및 주요 작업의 실행은 QCAT 보유 또는 스테이킹 상태를 기반으로 승인된다. QCAT이 없을 경우, 해당 실행은 구조적으로 수행될 수 없다.

Risk-weighted Execution

트랜잭션의 리스크 수준에 따라 요구되는 QCAT의 양이 달라진다.
일반 트랜잭션 → 낮은 QCAT 요구량,
고위험 트랜잭션 → 높은 QCAT 요구량
이를 통해 보안과 비용이 연동된 구조를 형성한다.

Node & Validator Participation

네트워크 참여자(노드, 검증자 등)는 QCAT을 스테이킹해야 하며, 부정행위 발생 시 슬래싱이 적용된다

Integrated Token Economy

QCAT은 단일 기능이 아닌, 플랫폼 전체를 관통하는 통합 경제 모델을 형성한다.

QCAT의 역할은 다음과 같이 정의된다:

Identity: 신원 생성 및 검증의 신뢰 담보
AI: 분석 및 실행 접근 통제
Payment: 결제 및 정산 수단
Execution: 실행 권한 통제 및 승인

이 구조를 통해 QCAT은 단순한 유틸리티 토큰을 넘어 플랫폼의 운영과 보안을 동시에 지탱하는 핵심 자산으로 기능한다.

Structural Necessity of QCAT

본 시스템은 QCAT 없이 핵심 기능이 작동하지 않도록 설계되어 있다.

이는 다음과 같은 원칙을 기반으로 한다:

선택적 사용(Optional Use)이 아닌 필수적 사용(Mandatory Use)
기능 보조가 아닌 실행 제어(Control of Execution)
단순 결제가 아닌 신뢰와 책임의 경제적 표현

결과적으로 QCAT은 단순한 토큰이 아니라, 플랫폼의 실행과 신뢰를 유지하기 위한 필수적인 구조적 요소다.